loss function

MSE : Mean-square error $\Rightarrow$ regression problem (o/p is real-valued)

↳ o/p activation is not softmax

$$J(w) = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$ → overall loss for all samples

$\hat{y}_i$: estimate → $\hat{y}_i = f(x_i, w)$

y_i : ground-truth

n : total number of samples

cross entropy $\Rightarrow$ classification problem

↳ o/p activation fn is softmax

$$J(w) = -\sum_{i}\sum_{c} y_i^c \log(\hat{y}_i^c)$$

c : no. of classes

Binary cross entropy (2 classes)

$$J(w) = -\sum_{i} \underbrace{(y_i \log(\hat{y}_i))}_{y_i=1} + \underbrace{(1-y_i)\log(1-\hat{y}_i)}_{y_i=0} \quad \Big| \quad y_i = \{0, 1\}$$

---

softmax :

$$\hat{y}_i = \frac{e^{x_i}}{\sum e^{x_i}}$$

$$\begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ \vdots \end{pmatrix} \quad \begin{pmatrix} 0.01 \\ 0.1 \\ 0.001 \\ 0.8 \\ 0.15 \\ \vdots \end{pmatrix} \Rightarrow \text{sum up to 1}$$

true labels

actual labels (o/p of softmax)

Optimization → Notation : $J(w)$: loss fn.

— Gradient descent → Batch gradient descent → loss over all data

$$\Delta w_j = -\eta \frac{\partial J}{\partial w_j} = -\eta \nabla_{w_j} J$$

η : step size

$$\boxed{\begin{aligned} w_j^{t+1} &= w_j^t + \Delta w_j \\ &= w_j^t - \eta \frac{\partial J}{\partial w_j} \end{aligned}}$$

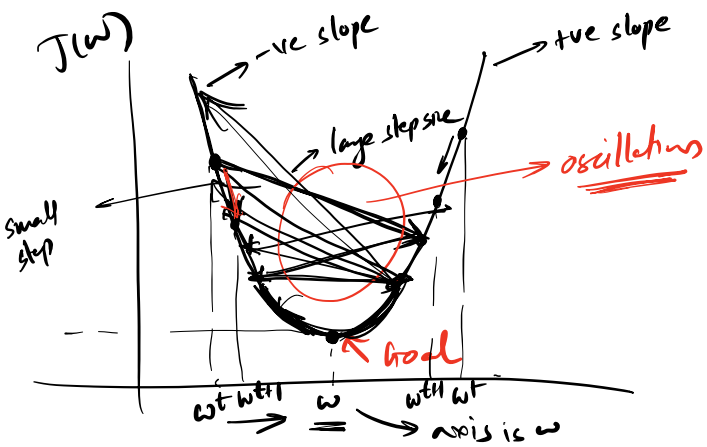
$$\boxed{J = \sum_{i=1}^{n} J_i(w_j)}$$

n : total no. of samples

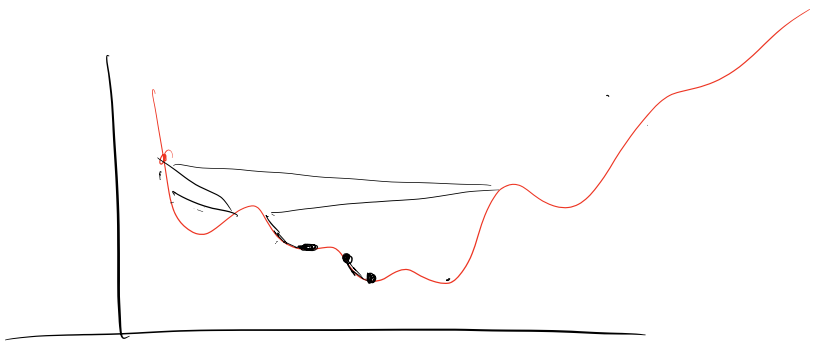
⇒ loss computed over the complete data

⇒ gradient computed over the complete data as the sum of gradients over individual samples

} Very memory and compute intensive



- computation
- step-size
  - small → slower convergence
  - large → faster convergence (initially) but can suffer from oscillations
- oscillations → may hinder convergence
  - ↘ may also help in jumping over a local minima



- Stochastic gradient descent

$$\Delta w_j = -\eta \frac{\partial J_i}{\partial w_j} = -\eta \nabla_{w_j} J_i$$

$\Rightarrow$ defining the loss for some sample i

$\Rightarrow$ weight updates are based on the gradient computed for that sample

- gradients for each sample will be noisier
- weight updates will need to repeated for different samples where the samples are selected randomly

$\rightarrow$ path to the minima will be more oscillatory

trade offs

Overall SGD is more efficient than GD

$\Downarrow$

- may take more time to converge
- But can avoid local minima

$\Leftarrow$ is better for non-convex loss fns

- mini-batch GD

$$J = \sum_{i=1}^{K} J_i(w_j)$$

K: subset of samples ($K \ll n$)

- less oscillations due to some avg of losses
- faster convergence | faster compute and less memory than BGD
- balance between stochastic and batch gradient descent

- Momentum

$$v^{t+1} = \beta v^t + (1-\beta)\nabla_{w^t} J$$

$v^0 = 0$

$v^1 = (1-\beta)\nabla_{w^0} J$

$v^2 = \beta v^1 + (1-\beta)\nabla_{w^1} J$

$= \beta(1-\beta)\nabla_{w^0} J + (1-\beta)\nabla_{w^1} J$

v^{t+1} includes the weighted avg from the previous gradients

$0 < \beta < 1$

$$\boxed{w^{t+1} = w^t - \eta v^{t+1}}$$

$\Rightarrow$ updates involve avg across gradients

weighted avg of the previous two gradients

stochastic or mini-batch

XGD: Avg is over samples

XGD + momentum: Avg is over time $\Rightarrow$ smoothy of gradients over time

$\hookrightarrow$ Better noise reduction
Gradients are smoothened in a progressive manner

- Adagrad

$$w^{t+1} = w^t - \frac{\eta}{\sqrt{G^t + \epsilon}} \nabla_{w^t} J$$

$\rightarrow$ step size

- Like momentum, we have some sort of integration over temporal gradients
- Unlike momentum, this is not used as an update to the weights

$\Downarrow$

But this is used in adapting the step size

G^t : sum of squared gradients from previous updates

$$G^t = \sum_{p=1}^{t-1} (\nabla_{w^p} J)^2$$

$\Rightarrow$ G^t keeps on $\uparrow$ over time

$\Rightarrow$ step size keeps on $\downarrow$ " "

- RMS prop

$$G^t = \gamma G^{t-1} + (1-\gamma)(\nabla_{w^t} J)^2$$

$$w^{t+1} = w^t - \frac{\eta}{\sqrt{G^t + \epsilon}} \nabla_{w^t} J$$

Adagrad: Step-size reduces agressively

RMS prop: Weighted combination of quadratic terms but where the weights can be adjusted to adjust the contribution of the current gradient terms

- Adam

$v^t = \beta_1 v^{t-1} + (1-\beta_1) \nabla_{w^{t-1}} J \Rightarrow$ moment

$G^t = \beta_2 G^{t-1} + (1-\beta_2)(\nabla_{w^t} J)^2 \Rightarrow$ RMS Prop

combination of momentum RMS prop

Bias correction: $\hat{v}^t = \frac{v^t}{\sqrt{1-\beta_1}}$, $\hat{G}_t = \frac{\hat{G}_t}{\sqrt{1-\beta_2}}$

$$\boxed{w_{t+1} = w_t - \eta \frac{v^t}{\sqrt{G^t + \epsilon}}} \Rightarrow$$

a) avg of gradients (momentum) across time

b) adaptive learning rate (RMSprop)

Batch GD
Stochastic GD
mini-batch GD
$\Rightarrow$ Variation over sample size

momentum $\Rightarrow$ Gradient weighted avg over time used as updates

adagrad
RMS prop
$\Rightarrow$ gradients are summed over time used for modulation of learning-rate (or step size)

Adam $\Rightarrow$ Combination of momentum and RMS prop